

Energy-efficient Codon Optimization on Thermodynamic Hardware

Andraž Jelinčič,^{1,*} Ross C. Walker¹

¹Extropic Corp., Boston, MA

andraz@extropic.ai, rwalker@extropic.ai

Abstract

The growing energy demand for computation is becoming increasingly unsustainable. Thermodynamic computing, which harnesses physical thermal fluctuations as a computational resource rather than suppressing them, offers orders-of-magnitude energy savings for probabilistic and combinatorial tasks. Pharmaceutical R&D, heavily reliant on computational optimization and sampling, is a natural application domain. Here we present what is, to our knowledge, the first concrete pharmaceutical application mapped to thermodynamic hardware with energy estimates grounded in prototype measurements. We reduce mRNA codon optimization, a combinatorial problem routinely solved in drug development, to sampling from an Ising model, making it directly executable on a thermodynamic sampling unit (TSU). Benchmarking three approaches (Potts sampling, Ising sampling, and a genetic algorithm baseline) on the SARS-CoV-2 spike protein, we find that all achieve comparable optimization quality (scores ~ 234 – 240), but energy estimates based on validated hardware models indicate that a TSU could solve this problem using approximately $10^6\times$ less energy than a conventional GPU. All code is released under an open-source license.

1 Introduction

Recent large-scale investments in artificial intelligence and computational workloads are placing increasing strain on the global energy infrastructure. Every year, U.S. companies spend an amount exceeding the inflation-adjusted cost of the Apollo program on AI-focused data centers. With demand projected to grow from 59 GW in 2025 to more than 120 GW by 2030 International Energy Agency [2024], there are concerns that the energy demand for data centers is becoming unsustainable. At the same time, demand driven increases in model sizes and deployment footprints continue to grow rapidly, creating an urgent need for more efficient computing paradigms.

Existing AI systems are optimized for GPU hardware which was originally designed for computer graphics and whose suitability for machine learning was only discovered accidentally decades later. Had different cost effective hardware architectures been available, algorithms may well have evolved in a different and possibly more energy-efficient direction. This interplay between algorithm research and hardware availability, known as the “hardware lot-

tery” Hooker [2021], entrenches hardware-algorithm pairings that may be far from optimal. Prudent planning calls for systematic exploration of alternative computing architectures Jelinčič et al. [2025].

Thermodynamic computing offers one such alternative Conte et al. [2019], Hylton [2020]. In nature, thermodynamic laws govern the behavior of large collections of interacting atoms, which inevitably tend to settle into stable arrangements with near-minimum energy. Many famous optimization algorithms including Langevin Monte Carlo, Simulated Annealing, and Diffusion Modeling, are inspired by (and in some cases, exactly simulating) this phenomenon. The goal of thermodynamic computing hardware is to target and accelerate algorithms of this type.

More specifically, this hardware can be used to perform an *analog simulation* of the physical dynamics that would arise in a system governed by a user-defined energy function. Allowing this simulation to relax toward thermal equilibrium produces samples from the corresponding *Boltzmann distribution*. This stands in sharp contrast to conventional computing, where GPUs and CPUs expend substantial energy suppressing thermal noise, an intrinsic prop-

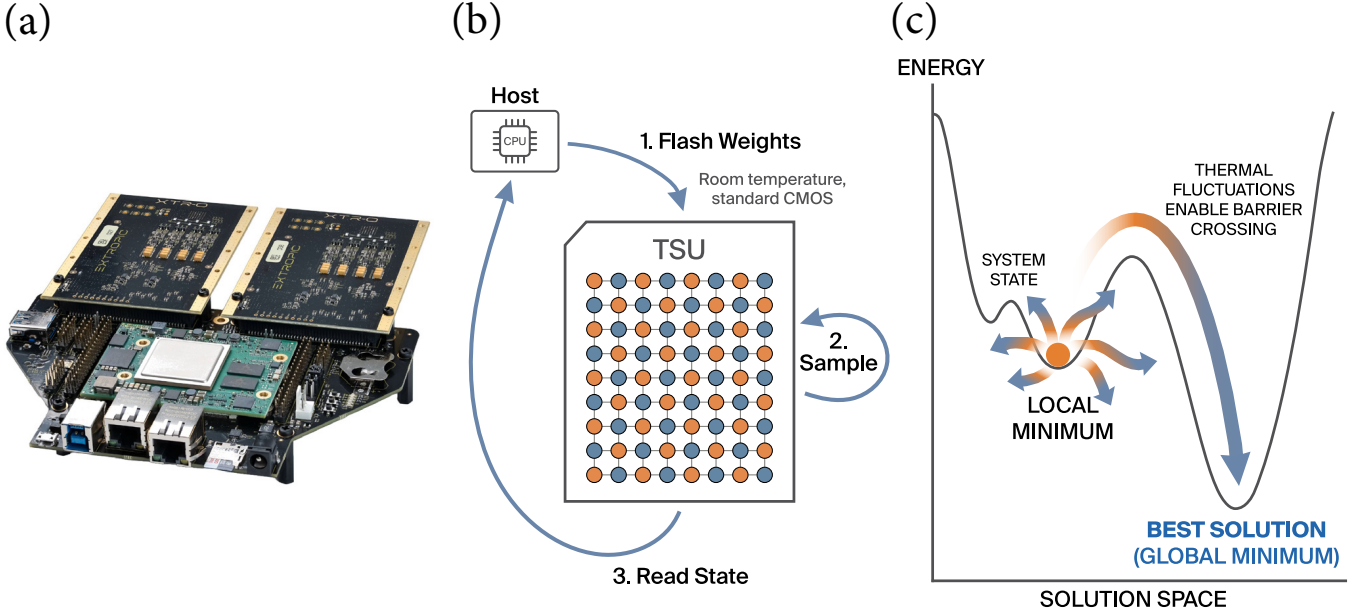


Figure 1: **Thermodynamic computing overview.** (a) A picture of a system containing two of Extrinsic’s prototype thermodynamic computing chips. These chips implement probabilistic computing primitives and are used to confirm the correct operation of these primitives and to measure their energy consumption. (b) Schematic of a thermodynamic sampling unit (TSU). The chip contains a grid of interconnected probabilistic bits (p-bits), two-colored for parallel block Gibbs updates. The host programs weights, the chip performs rapid sampling, and the host reads back the result. (c) Stochastic optimization on an energy landscape. Thermal fluctuations enable the system to explore broadly and escape local minima, converging toward low-energy solutions.

erty of all integrated circuits, in order to guarantee deterministic operation. When such hardware is then used to run inherently probabilistic algorithms, the result is a fundamental inefficiency: considerable energy is spent enforcing perfect determinism, only for the software to immediately re-inject randomness through pseudo-random number generation. Thermodynamic hardware circumvents this logic entirely, harnessing the very thermal fluctuations that conventional processors expend energy to suppress, and thereby performing natively what conventional hardware must laboriously simulate Jelinčič et al. [2025].

The physical embodiment of this approach is the thermodynamic sampling unit (TSU): a programmable chip fabricated in standard CMOS, operating at room temperature, that contains arrays of probabilistic computing elements connected in a sparse, locally-connected graph (Fig. 1). A TSU is built from a family of primitive sampling circuits (Table 1), each of which natively samples from a different distribution class: p-bits (Bernoulli), p-dits (categorical), p-modes (Gaussian), and p-MoGs (mixture of Gaussians). By programming

the interaction weights between these elements, the chip’s thermal fluctuations explore the corresponding Boltzmann distribution. This work uses p-bits for the Ising formulation and, prospectively, p-dits for the native Potts formulation. A TSU operates in a flash-sample-read cycle: a host computer programs the interaction weights, the chip performs many rounds of Gibbs sampling at extreme speed and low energy, and the host reads back the resulting state.

Recent work has demonstrated that a device based on this architecture could achieve performance parity with GPUs on a generative image benchmark while using approximately $10,000\times$ less energy per sample Jelinčič et al. [2025]. Real world measurements from prototype hardware have validated the underlying energy models. Prototype hardware is already in production and a TSU capable of running the algorithms described in this paper is projected to be commercially available within two years.

Table 1: Probabilistic primitives of a TSU.

Primitive	Distribution	Hardware implementation
p-bit	Bernoulli (binary)	Subthreshold transistor RNG with sigmoidal bias
p-dit	Categorical (K -state)	Softmax sampling circuit
p-mode	Gaussian (continuous)	Analog noise source with programmable mean/variance
p-MoG	Mixture of Gaussians	Array of coupled p-modes

Pharmaceutical applications. Pharmaceutical research and development is a natural application domain for thermodynamic computing. Drug discovery relies heavily on computational optimization and sampling. For example: conformational search, binding affinity estimation, molecular design, protein folding and sequence optimization all reduce to energy minimization or probabilistic inference over discrete or continuous variables. These are precisely the problems for which thermodynamic hardware is designed. Conventional hardware such as GPUs, optimized for deterministic matrix multiplication, is inherently inefficient for these natively probabilistic workloads. Nevertheless, to our knowledge, no prior work has demonstrated the potential applications of thermodynamic computing to discrete optimization problems in drug design nor provided energy estimates grounded in real measurements.

In this work, we provide such a demonstration using mRNA codon optimization as an example real world workload. In all living organisms, proteins are synthesized by ribosomes that read messenger RNA (mRNA) in triplets called *codons*, each coding for one amino acid. Because the genetic code is degenerate, with 61 codons encoding for only 20 amino acids, most amino acids can be specified by two to six *synonymous* codons. Although synonymous codons encode the same amino acid, they are not functionally equivalent: codon sequence matters and different organisms use them at different frequencies (a phenomenon known as *codon bias*). This affects translation speed, mRNA stability, and ultimately protein expression levels Gustafsson et al. [2004], Quax et al. [2015], Brulé and Grayhack [2017]. When expressing a recombinant protein in a host organism, choosing codons that match the host’s preferences and selecting codon sequences for optimum translation speed and stability, in a process referred to as *codon optimization*, can dramatically improve expression. For a more comprehensive review of codon optimization and its biological underpinnings, we refer the reader

to Gustafsson et al. [2004] and Quax et al. [2015].

Codon optimization is a combinatorial optimization problem that is routinely solved in pharmaceutical and biotechnology pipelines, with direct relevance to mRNA therapeutics, including COVID-19 vaccines Polack et al. [2020] and protein replacement therapies Fox et al. [2021]. We formulate the problem as a Potts model (a probabilistic graphical model with categorical variables) and compile it to an Ising model via domain-wall encoding Chancellor [2019], making it executable on binary (p-bit) hardware. We show that all three approaches, Potts sampling, Ising sampling, and a carefully tuned genetic algorithm (GA) baseline, achieve near-identical optimization quality on the SARS-CoV-2 spike protein. The decisive differentiator is energy consumption: estimates based on validated hardware models indicate that a TSU could solve this problem using between 10^5 – 10^9 times less energy than a conventional GPU. All code is built on the THRML library Jelinčič et al. [2025] and available under an open-license at github.com/extropic-ai/thrml.

2 Results

2.1 Problem setup

We benchmark all methods using the SARS-CoV-2 spike protein, a biologically relevant sequence of $L = 1,273$ amino acids (requiring 3,819 mRNA nucleotides), using *Escherichia coli* K-12 as the host organism. The energy function to be minimized is a weighted combination of three terms (see Section 4.1 and Fig. 3a): a codon usage penalty ($w_f = 0.1$) that favors codons commonly used in the host, a GC content term ($w_{GC} = 1$) that penalizes deviation from a target GC fraction of $\rho_T = 0.5$, and a repeat penalty ($w_R = 0.1$) that discourages long runs of identical nucleotides across codon boundaries. These weights follow the formulation of Fox et al. [2021] and reflect

Table 2: Optimization quality on the SARS-CoV-2 spike protein ($L = 1,273$) for two parameter settings (see Section 4.1). We report the final energy (mean $\pm$ std across 512 independent chains) and the best score found across all chains. Lower scores indicate better solutions. The number of sweeps chosen is the minimal number of sweeps that achieves a score close to the best score achieved by all methods.

Setting	Method	Mean score $\pm$ std	Best score
Standard (Fox)	Potts model (10 sweeps)	234.6 $\pm$ 0.2	234.0
	Ising model (2×10^4 sweeps, DWE)	243.0 $\pm$ 0.8	240.5
	Genetic algorithm (tuned)	234.8 $\pm$ 0.4	233.9
Hard	Potts model (10 sweeps)	445.2 $\pm$ 1.0	444.0
	Ising model (4×10^4 sweeps, DWE)	452.8 $\pm$ 1.1	449.2
	Genetic algorithm (tuned)	446.4 $\pm$ 1.0	444.4

commonly used values in codon optimization practice.

We evaluate three optimization methods. First, a **Potts model** sampler that operates directly on categorical codon variables (see Section 4.2 and Fig. 3b) using simulated annealing with block Gibbs updates (10 sweeps). Second, an **Ising model** sampler that encodes the categorical variables into binary spins via domain-wall encoding (see Section 4.4 and Fig. 3b,c) and performs simulated annealing with simultaneous penalty ramping (2×10^4 sweeps). Third, a **genetic algorithm** (GA) baseline reimplemented from Fox et al. [2021] with more carefully tuned parameters (population 200, 1,000 generations, mutation rate 0.003; see Section 4.6 for sensitivity analysis).

2.2 Optimization quality

Table 2 summarizes the optimization quality achieved by each method under both the standard weights from Fox et al. [2021] and the harder parameter setting described in Section 4.1. Under both settings, all three methods produce solutions of comparable quality. The Potts model and GA achieve essentially identical performance, while the Ising model scores slightly higher, which is expected given the overhead of the binary encoding.

The key observation is that all methods effectively achieve the same optimization quality under both parameter settings, within $\sim 4\%$ of each other, making the solution quality interchangeable and the energy consumption the decisive differentiator. All energy estimates below are for the standard parameter setting from Fox et al. [2021].

2.3 Energy consumption

We estimate the energy required to solve the codon optimization problem on thermodynamic hardware and compare it to conventional GPU execution (see Section 4.7 for full methodology). The TSU energy model, validated against prototype hardware measurements Jelinčič et al. [2025], estimates $E_{\text{cell}} \approx 1.3$ fJ per spin per Gibbs step.

Ising chip. The Ising model uses $N = 3,147$ binary spins (p-bits) and $K = 2 \times 10^4$ Gibbs iterations. The estimated energy is:

$$\begin{aligned}
 E_{\text{Ising}} &= K \times N \times E_{\text{cell}} \\
 &= 2 \times 10^4 \times 3,147 \times 1.3 \times 10^{-15} \text{ J} \approx 82 \text{ nJ}.
 \end{aligned}
 \tag{1}$$

Potts chip (projected). Based on a physical model of categorical (p-dit) hardware extrapolated from prototype measurements, a Potts chip would consume approximately 0.084 nJ per Gibbs sweep for $L = 1,273$ nodes, giving $10 \times 0.084 = 0.84$ nJ for the full optimization. We note that Potts (p-dit) hardware does not yet exist; the Ising (p-bit) TSU is the near-term target. An example of how a Potts model could be implemented in hardware can be found in Whitehead et al. [2023]. We based our energy estimates both on the measurements from our prototype and a design similar to the one proposed in Whitehead et al. [2023].

GPU energy. For GPU comparison on an NVIDIA RTX 3090 (350 W TDP), we bracket the

energy between an idealized lower bound (peak FLOP throughput) and a measured upper bound (wall-clock time $\times$ TDP); see Section 4.7. The Potts model requires $\sim 1.3 \times 10^6$ FLOPs (10 sweeps), the Ising model $\sim 2 \times 10^9$ FLOPs (2×10^4 sweeps), and the GA $\sim 6 \times 10^9$ FLOPs—roughly 4,600 $\times$ more than the Potts sampler for the same solution quality.

Table 3 and Fig. 2 present the complete energy comparison.

Table 3: Energy per single-chain optimization run on the spike protein ($L = 1,273$), standard (Fox) weights. Ratios relative to Ising chip.

Configuration	Energy	Ratio v. Ising
Potts (proj.)	0.84 nJ	0.01 $\times$
Ising	82 nJ	1 $\times$ (ref.)
<i>GPU — idealised (FLOPs $\times$ peak eff.)</i>		
Potts, 3090	13 μ J	160 $\times$
Ising, 3090	20 mJ	2.4×10^5
GA, 3090	59 mJ	7.2×10^5
<i>GPU — measured (wall time $\times$ TDP)</i>		
Potts, 3090	245 mJ	3.0×10^6
Ising, 3090	~ 84 J	$\sim 10^9$
GA, 3090	21 J	2.6×10^8

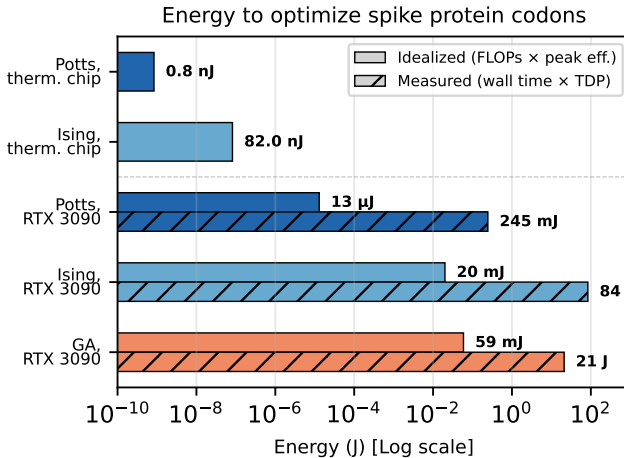


Figure 2: **Energy comparison across methods and hardware.** All methods achieve comparable optimization quality (score ~ 235 – 243); the differentiator is energy consumption, which spans approximately 10 orders of magnitude. Thermodynamic chip estimates are based on validated hardware models Jelinčić et al. [2025].

Several features of this comparison are noteworthy. First, even at idealized peak efficiency (a generous lower bound that credits the GPU with 100% utilization) the thermodynamic chip is between 160 and 2.4×10^5 times more efficient than a GPU running the same algorithm (Potts and Ising respectively). Second, realistic GPU utilization is far below peak: the Potts model’s $\sim 10^6$ FLOPs represent a tiny fraction of the RTX 3090’s throughput, and execution time is dominated by kernel launch overhead. The measured GPU energy therefore exceeds the idealized estimate by several orders of magnitude. Third, the GA, which appears to be the most common algorithm used for codon optimization [Fox et al., 2021], requires $\sim 4,600$ times more FLOPs than the Potts sampler for the same solution quality, compounding the hardware efficiency gap. The combined advantage of a more efficient algorithm running on more efficient hardware yields energy savings of 10^5 – 10^9 times relative to a GPU running the GA.

2.4 Scaling considerations

Solving the codon optimization problem for the spike protein requires 3,147 spins in the Ising formulation, well within the capacity of near-term TSU designs ($\sim 250,000$ p-bits). This leaves substantial headroom for larger proteins, longer mRNA constructs, or more complex energy functions incorporating additional biological objectives. The chain-like graph structure of the problem, with a maximum node degree of 12, maps naturally to the sparse, locally-connected topology of a TSU.

At a random number generator (RNG) decorrelation time of ~ 100 ns (measured from prototype hardware Jelinčić et al. [2025]), a full Gibbs step on a 4-colorable graph takes ~ 400 ns. The 2×10^4 -step Ising optimization would therefore complete in approximately 8 ms on a TSU which is more than fast enough for integration into real-time computational pipelines.

3 Discussion

This work demonstrates that a real world pharmaceutical optimization problem can be efficiently mapped to thermodynamic hardware, with energy savings of five to nine orders of magnitude compared to conventional computing (Table 3). By taking a

concrete pharmaceutical application from problem formulation through hardware mapping to energy estimation grounded in prototype measurements, it provides a template for evaluating thermodynamic computing on applied problems.

The Potts formulation is algorithmically superior to the Ising encoding: it uses fewer variables (1,273 categorical vs. 3,147 binary), requires fewer sampling sweeps (10 vs. 2×10^4), and achieves slightly better scores. However, building Ising (p-bit) hardware is substantially simpler than building Potts (p-dit) hardware. A TSU capable of running the Ising formulation could be commercially available within one to two years, while categorical hardware remains further out. Crucially, the $\sim 100\times$ energy gap between the projected Potts chip and the Ising chip is modest compared to the 10^5 – 10^9 times advantage that *both* enjoy over GPUs. This favors the Ising formulation as the practical near-term path.

We acknowledge several limitations: The codon optimization objective used here is a simplified version of what production pipelines employ; more involved formulations may include terms for mRNA secondary structure, CpG dinucleotide content, and untranslated region effects Fox et al. [2021]. These can be incorporated as additional local or pairwise energy terms without fundamentally changing the graph structure or hardware requirements; The energy estimates for the Potts chip are projections based on extrapolation from prototype measurements, while the Ising estimates rest on a validated physical model with close agreement to experimental data Jelinčič et al. [2025].; Finally, codon optimization itself is not the computational bottleneck in drug development, but it does serve as a concrete, accessible proof of concept that the pipeline from problem formulation through hardware mapping to energy estimation is viable.

More broadly, thermodynamic computing is not limited to codon optimization. The same hardware that solves Ising models for codon sequences can accelerate conformational search, binding affinity estimation, molecular generation, and other sampling-intensive drug discovery workloads. The key insight is that drug discovery problems naturally decompose into energy minimization over discrete or continuous variables, precisely the class of problems for which thermodynamic hardware is designed. The rapid growth of mRNA therapeutics, from COVID-19 vac-

cines Polack et al. [2020] to cancer immunotherapies and protein replacement therapies, ensures that codon optimization itself will remain an important step in drug development pipelines, and one where energy-efficient computation at scale becomes increasingly valuable. As thermodynamic hardware matures, we anticipate it will unlock efficient computation across a broad range of drug discovery and similar scientific applications.

Code availability. All code for this work is open-source at https://github.com/extropic-ai/codon_opt, built on the THRML library Jelinčič et al. [2025].

4 Methods

4.1 Codon optimization problem

Given a protein sequence of L amino acids $a_1, \dots, a_L$, the codon optimization problem is to select a synonymous codon $c_p \in S_p$ at each position p to produce an mRNA sequence with desirable expression properties in a target host organism Gustafsson et al. [2004], Quax et al. [2015], Brulé and Grayhack [2017]. The set S_p contains all codons encoding amino acid a_p , with $|S_p| \in \{1, 2, 3, 4, 6\}$ depending on the amino acid.

The objective is to minimize the energy function:

$$E(c_1, \dots, c_L) = w_f \sum_{p=1}^L u(c_p) + w_R \sum_{p=1}^{L-1} r(c_p, c_{p+1}) + w_{GC} \left(\frac{1}{3L} \sum_{p=1}^L g(c_p) - \rho_T \right)^2 \quad (2)$$

where:

- The *codon usage* term $u(c) = |\log(f(c)/f_{\max}^{(a)})|$ measures how rare codon c is in the host relative to the most common codon for its amino acid, using host-specific codon frequency tables Nakamura et al. [2000]. This term is *unary*: it depends on each position independently.
- The *GC content* term penalizes deviation of the overall GC fraction from a target ρ_T , where $g(c) \in \{0, 1, 2, 3\}$ counts the G and C nucleotides in codon c . This term is *global*: it couples all positions.

- The *repeat penalty* $r(c_p, c_{p+1}) = m(c_p, c_{p+1})^2 - 1$ penalizes long runs of identical nucleotides across codon boundaries, where m is the length of the longest contiguous run in the 6-character concatenation. This term is *pairwise*: it depends on adjacent positions only.

We follow the formulation and weights of Fox et al. [2021]: $w_f = 0.1$, $w_{GC} = 1$, and $w_R = 0.1$. These reflect commonly used values in codon optimization practice, and all energy estimates in Section 2.3 are based on this setting. We additionally report optimization quality for a harder parameter setting ($w_f = 0.1$, $w_{GC} = 2 \times 10^4$, $w_R = 0.2$) in which the three energy terms are approximately equally balanced, making the problem substantially more difficult. Under the standard (Fox) weights the GC content term is negligible relative to the other two, which makes the problem easier; indeed all three methods require more iterations to converge under the hard weights than under the standard ones (see Table 2).

4.2 Potts model formulation

The codon optimization problem maps naturally to a Potts model Wu [1982]: each amino acid position p becomes a categorical random variable X_p with $|S_p|$ states, one for each synonymous codon. All variables are padded to $K_{\max} = 6$ states, with invalid states receiving a large positive bias that effectively excludes them.

The Potts energy decomposes into unary biases $h_p(k)$ (from the codon usage term and validity constraints) and pairwise interactions $J_p(k, k')$ (from the repeat penalty between adjacent positions p and $p + 1$):

$$E(\mathbf{x}) = \sum_{p=1}^L h_p(x_p) + \sum_{p=1}^{L-1} J_p(x_p, x_{p+1}). \quad (3)$$

Since each position interacts only with its immediate neighbors, the resulting graph is a chain, which is 2-colorable. This enables efficient parallel block Gibbs sampling: conditioned on odd-indexed positions, all even-indexed positions are conditionally independent and can be sampled simultaneously via a softmax distribution over their K states, and vice versa. One full Gibbs sweep consists of updating the even block and then the odd block.

4.3 Adaptive GC coefficient

The GC content term in Eq. (2) couples all positions through a global quadratic constraint. Expanding this term directly would require all-to-all pairwise interactions, which is incompatible with the sparse connectivity of a TSU. All-to-all connectivity also increases the chromatic number of the graph to be equal to the total number of nodes, which completely prevents sampling blocks of nodes in parallel. Instead, we approximate it with an adaptive linear term: a per-chain coefficient λ that modifies the unary bias of each codon proportionally to its GC count:

$$\tilde{h}_p(k) = h_p(k) + \lambda \cdot g(c_p^{(k)}). \quad (4)$$

At each annealing step, λ is updated based on the current GC fraction error:

$$\lambda \leftarrow \lambda - \eta \cdot (\rho_{\text{current}} - \rho_T) \quad (5)$$

where $\eta > 0$ is an adaptation rate. If the current GC fraction exceeds the target, λ decreases (penalizing high-GC codons); if below, λ increases. To prevent the linear approximation from dominating, λ is clamped to never exceed the gradient of the true quadratic GC penalty, $\lambda_{\text{true}} = -2w_{GC}(\rho_{\text{current}} - \rho_T)/(3L)$.

This scheme is naturally compatible with the TSU’s flash-sample-read cycle: λ is updated on the host between sampling bursts and folded into the biases before the next flash. This incurs no additional cost, since new weights would regardless need to be flashed to the TSU due to annealing.

4.4 Ising embedding via domain-wall encoding

Binary spin hardware (with p-bits) is simpler to build and closer to commercial availability than categorical hardware (with p-dits). To make the codon optimization problem executable on a binary TSU, we compile the Potts model into an Ising model using domain-wall encoding (DWE) Chancellor [2019], Berwald et al. [2022], Chen et al. [2021].

A K -state Potts variable is represented by $K-1$ binary spins $s_1, \dots, s_{K-1} \in \{-1, +1\}$ arranged in a chain. Valid states follow a thermometer pattern: state k corresponds to $s_1 = \dots = s_k = +1$ and

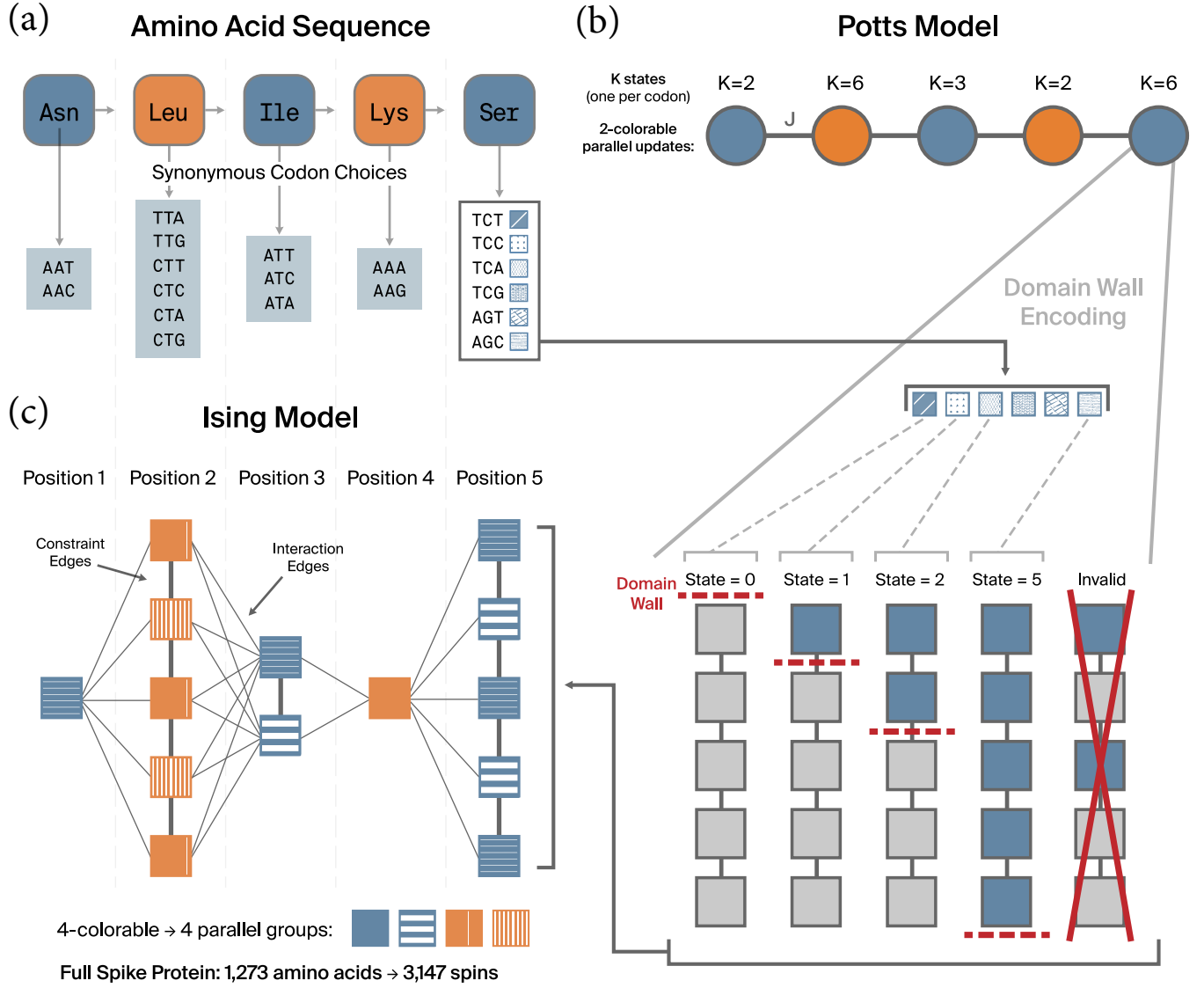


Figure 3: **From codons to Ising spins.** (a) The codon optimization problem: each amino acid in a protein sequence is encoded by a triplet codon, with multiple synonymous options. The goal is to choose codons that optimize usage frequency, GC content, and repeat avoidance. (b) Potts model representation with domain-wall encoding. Each amino acid position becomes a categorical variable in a chain graph (top); the chain is 2-colorable, enabling parallel block Gibbs updates. Each K -state Potts variable is encoded as $K-1$ binary spins in a thermometer pattern (bottom), where the domain wall position encodes the categorical state. (c) Full Ising graph for a short protein segment. Intra-position constraint edges enforce validity; inter-position edges encode codon interactions. The graph is 4-colorable for parallel updates. The spike protein maps to 3,147 spins.

$s_{k+1} = \dots = s_{K-1} = -1$. The “domain wall”—the boundary between the $+1$ and -1 regions—encodes the Potts state. Invalid configurations (with a -1 followed by a $+1$) are penalized by ferromagnetic nearest-neighbor couplings of strength $P/4$ within each position’s spin chain, adding $K-2$ constraint edges per position.

DWE offers several advantages over the more common one-hot encoding Lucas [2014]. First, the constraint graph is a chain ($K-2$ edges) rather than a clique ($K(K-1)/2$ edges), yielding a much sparser graph. Second, the chain constraint is 2-colorable, which at most doubles the chromatic number of the overall graph—unlike one-hot, which requires K colors for its penalty clique, which can therefore increase the chromatic number of the graph K -fold. Since the chromatic number of the graph determines how many sampling steps are required per Gibbs iteration, the one-hot encoding is typically $K/2$ -times slower than DWE per Gibbs iteration, on top of possibly also requiring a far greater number of Gibbs iterations due to its slow mixing. Third, mixing at the domain wall boundary is independent of the penalty strength P : the ferromagnetic contributions from the two neighbors cancel, so the Gibbs conditional depends only on the bias landscape. This eliminates the exponential mixing slowdown Levin et al. [2009] that plagues one-hot encoding at large P Berwald et al. [2022]. Fourth, DWE has been proven optimal in the number of binary variables for quadratic interactions Berwald et al. [2022].

Potts biases are compiled to Ising fields via first differences ($b_i = (h_i - h_{i-1})/2$ plus boundary corrections), and Potts pairwise interactions become Ising couplings via mixed second differences ($J_{(p,i),(q,j)} = (V_{ij} - V_{i-1,j} - V_{i,j-1} + V_{i-1,j-1})/4$); see Supplementary Note 1 for the full derivation.

GC-sorted codon ordering. In DWE, a single spin flip moves the Potts variable to an adjacent state ($k \rightarrow k \pm 1$). We order codons at each position by ascending GC count (with alphabetical tiebreaking), so that a single spin flip changes GC content by at most 1. This creates a smooth landscape for domain wall dynamics and ensures compatibility with the adaptive GC scheme.

The resulting Ising model for the spike protein contains $N = 3,147$ spins (an average of 2.47 per amino acid position), with a maximum node degree of 12

and only local connectivity. The graph is 4-colorable using a decomposition based on position parity and spin-index parity.

4.5 Simulated annealing and P-ramping

Both the Potts and Ising models use simulated annealing with a log-spaced inverse temperature schedule:

$$\beta_t = \beta_{\min} \cdot \left(\frac{\beta_{\max}}{\beta_{\min}} \right)^{t/T} \quad (6)$$

for $t \in \{0, \dots, T\}$. At each temperature step, the host updates β , recomputes the adaptive GC coefficient, flashes new weights to the TSU, and runs a fixed number of Gibbs sweeps.

For the Ising model, we simultaneously ramp the constraint penalty P on an independent log-spaced schedule from $P_{\min}$ to $P_{\max}$. Low P early in annealing permits invalid-state (defect) transitions that provide shortcuts around energy barriers in the bias landscape: when a valid intermediate state has energy exceeding P , the defect route is energetically cheaper. High P at the end enforces strict constraint satisfaction. Since weights are recomputed at each annealing step anyway (for β and the adaptive GC coefficient), changing P adds no extra cost.

4.6 Genetic algorithm baseline

Genetic algorithms are widely used for codon optimization Fox et al. [2021], Sandhu et al. [2008]. As a baseline, we reimplemented the GA from Fox et al. [2021] in JAX, with full JIT compilation of the generation loop and multi-chain parallelism. After systematic parameter tuning, the best configuration uses a population of 200, 1,000 generations, 10 elite and 2 lucky survivors per generation, and a mutation rate of 0.003.

The GA’s performance is sensitive to parameter choice. The default mutation rate of 0.05 from Fox et al. yields substantially worse scores (~ 550 – 600), suggesting that the crossover operator provides sufficient exploration and high mutation rates are counterproductive, disrupting good partial solutions faster than selection can preserve them.

4.7 Energy estimation methodology

TSU energy model. We use the energy model from Jelinčič et al. [2025] (Appendix E), which provides a physical model of an all-transistor Boltzmann machine Gibbs sampler. The energy per spin per Gibbs step is $E_{\text{cell}} \approx 1.3$ fJ, which includes contributions from the RNG (~ 350 aJ), biasing circuitry, clocking, and inter-cell communication. This model captures all central functional units of the hardware and has been validated against measurements from a prototype chip, with agreement within an order of magnitude.

For the Ising chip, the total energy is $E = K \times N \times E_{\text{cell}}$, where K is the number of Gibbs iterations and N is the number of spins. For the Potts chip, we use a physical model of p-dit hardware extrapolated from the same prototype measurements, yielding ~ 0.084 nJ per Gibbs sweep for $L = 1,273$ Potts nodes.

Our Ising model has a maximum degree of 12 with only local connectivity, closely matching the setup analyzed in Jelinčič et al. [2025]. The codon problem (3,147 spins) fits comfortably within the capacity of near-term TSU designs ($\sim 250,000$ p-bits).

GPU energy model. We estimate GPU energy on an NVIDIA RTX 3090 (350 W TDP, 35.6 TFLOPS FP32 peak) using two methods. The *idealized* estimate computes total FLOPs divided by peak FLOP/s, multiplied by TDP. This represents a lower bound, as no real workload sustains 100% utilization. The *measured* estimate multiplies the wall-clock time of our JIT-compiled JAX implementation by TDP, giving an upper bound since the GPU may not draw full TDP power during execution. FLOP counts were estimated by analyzing the per-step computational requirements of each algorithm (see Supplementary Note 2 for details).

References

- Jesse Berwald, Nicholas Chancellor, and Raouf Dridi. Understanding domain-wall encoding theoretically and experimentally. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381(2241), December 2022. ISSN 1471-2962. doi: 10.1098/rsta.2021.0410.
- Christina E. Brulé and Elizabeth J. Grayhack. Synonymous codons: Choose wisely for expression. *Trends in Genetics*, 33(4):283–297, April 2017. ISSN 0168-9525. doi: 10.1016/j.tig.2017.02.001.
- Nicholas Chancellor. Domain wall encoding of discrete variables for quantum annealing and QAOA. *Quantum Science and Technology*, 4(4):045004, August 2019. ISSN 2058-9565. doi: 10.1088/2058-9565/ab33c2.
- Jie Chen, Tobias Stollenwerk, and Nicholas Chancellor. Performance of domain-wall encoding for quantum annealing. *IEEE Transactions on Quantum Engineering*, 2:1–14, 2021. ISSN 2689-1808. doi: 10.1109/TQE.2021.3094280.
- Tom Conte, Erik DeBenedictis, Natesh Ganesh, Todd Hylton, John Paul Strachan, R. Stanley Williams, Alexander Alemi, Lee Altenberg, Gavin Crooks, James Crutchfield, Lidia del Rio, Josh Deutsch, Michael DeWeese, Khari Douglas, Massimiliano Esposito, Michael Frank, Robert Fry, Peter Harsha, Mark Hill, Christopher Kello, Jeff Krichmar, Suhas Kumar, Shih-Chii Liu, Seth Lloyd, Matteo Marsili, Ilya Nemenman, Alex Nugent, Norman Packard, Dana Randall, Peter Sadowski, Narayana Santhanam, Robert Shaw, Adam Stieg, Elan Stopnitzky, Christof Teuscher, Chris Watkins, David Wolpert, Joshua Yang, and Yan Yufik. Thermodynamic computing, 2019. URL <https://arxiv.org/abs/1911.01968>.
- Dillion M. Fox, Kim M. Branson, and Ross C. Walker. mRNA codon optimization with quantum computers. *PLOS ONE*, 16(10):e0259101, October 2021. ISSN 1932-6203. doi: 10.1371/journal.pone.0259101.
- Claes Gustafsson, Sridhar Govindarajan, and Jeremy Minshull. Codon bias and heterologous protein expression. *Trends in Biotechnology*, 22(7):346–353, July 2004. ISSN 0167-7799. doi: 10.1016/j.tibtech.2004.04.006.
- Sara Hooker. The hardware lottery. *Communications of the ACM*, 64(12):58–65, November 2021. ISSN 1557-7317. doi: 10.1145/3467017.
- Todd Hylton. Thermodynamic computing: An intellectual and technological frontier. In *IS4SI 2019 Summit*, volume 47 of *Proceedings*, page 23. MDPI, June 2020. doi: 10.3390/proceedings2020047023.
- International Energy Agency. Electricity 2024: Analysis and forecast to 2026. Technical report, IEA,

- Paris, 2024. URL <https://www.iea.org/reports/electricity-2024>. Licence: CC BY 4.0.
- Andraž Jelinčič, Owen Lockwood, Akhil Garlapati, Peter Schillinger, Isaac Chuang, Guillaume Verdon, and Trevor McCourt. An efficient probabilistic hardware architecture for diffusion-like models, 2025. URL <https://arxiv.org/abs/2510.23972>.
- David A. Levin, Yuval Peres, and Elizabeth L. Wilmer. *Markov Chains and Mixing Times*. American Mathematical Society, Providence, RI, 2009. ISBN 978-0-8218-4739-8.
- Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014. ISSN 2296-424X. doi: 10.3389/fphy.2014.00005.
- Y. Nakamura, T. Gojobori, and T. Ikemura. Codon usage tabulated from international DNA sequence databases: status for the year 2000. *Nucleic Acids Research*, 28(1): 292, January 2000. ISSN 1362-4962. doi: 10.1093/nar/28.1.292.
- Fernando P. Polack, Stephen J. Thomas, Nicholas Kitchin, Judith Absalon, Alejandra Gurtman, Stephen Lockhart, John L. Perez, Gonzalo Pérez Marc, Edson D. Moreira, Cristiano Zerbini, Ruth Bailey, Kena A. Swanson, Satrajit Roychoudhury, Kenneth Koury, Ping Li, Warren V. Kalina, David Cooper, Robert W. Frenck, Laura L. Hammitt, Özlem Türeci, Haylene Nell, Axel Schaefer, Serhat Ünal, Dina B. Tresnan, Susan Mather, Philip R. Dormitzer, Uğur Şahin, Kathrin U. Jansen, and William C. Gruber. Safety and efficacy of the BNT162b2 mRNA Covid-19 vaccine. *New England Journal of Medicine*, 383(27):2603–2615, December 2020. ISSN 1533-4406. doi: 10.1056/NEJMoa2034577.
- Tessa E. F. Quax, Nico J. Claassens, Dieter Söll, and John van der Oost. Codon bias as a means to fine-tune gene expression. *Molecular Cell*, 59(2):149–161, July 2015. ISSN 1097-2765. doi: 10.1016/j.molcel.2015.05.035.
- Kuljeet Singh Sandhu, Sunil Pandey, Souvik Maiti, and Beena Pillai. GASCO: Genetic algorithm simulation for codon optimization. *In Silico Biology*, 8(2):187–192, 2008. ISSN 1434-3207. doi: 10.3233/ISB-00354.
- William Whitehead, Zachary Nelson, Kerem Y. Camsari, and Luke Theogarajan. CMOS-compatible Ising and Potts annealing using single-photon avalanche diodes. *Nature Electronics*, 6(12):1009–1019, November 2023. ISSN 2520-1131. doi: 10.1038/s41928-023-01065-0.
- F. Y. Wu. The Potts model. *Reviews of Modern Physics*, 54(1):235–268, January 1982. ISSN 0034-6861. doi: 10.1103/RevModPhys.54.235.